

湖北大学 课程考试试题纸

课程名称: 算法设计与分析 (A、B 卷)
 考试方式: 闭卷 (开卷、闭卷) 印刷份数: _____
 学 院: 计算机与信息工程学院 任课教师: 马传香、金红
 专业年级: 计算机科学与技术 2020 级、软件工程 2020 级、信息安全 2020 级

题号	一	二	三	四	五	六	七	八	总分	阅卷教师
得分										

得分	
----	--

一、计算复杂性分析 (每题 10 分, 共 30 分)

1、若 $f(n) = \log_2^2 n$, $g(n) = \log_2 n$, 请分析确定 $f(n) = O(g(n))$ 或 $f(n) = \Omega(g(n))$ 或 $f(n) = \theta(g(n))$, 并简述理由。

解:

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} &= \lim_{n \rightarrow \infty} \frac{\log_2^2 n}{\log_2 n} \\ &= \lim_{n \rightarrow \infty} \log_2 n \\ &= +\infty \end{aligned}$$

所以, $f(n) = \Omega(g(n))$

2、求解梵塔问题的递归算法如下，分析其时间复杂度。

```

void Hanoi(int n,char x, char y,char z)
{
    if(n==1)
        printf("将盘片%d 从%c 搬到%c\n",n,x,z);
    else
    {
        Hanoi(n-1,x,z,y);
        printf("将盘片%d 从%c 搬到%c\n",n,x,z);
        Hanoi(n-1,y,x,z);
    }
}

```

解：

设调用 $\text{Hanoi}(n,x,y,z)$ 的执行时间为 $T(n)$ ，由其执行过程得到以下求执行时间的递归关系（递推关系式）

$$T(n) = O(1) \quad \text{当 } n = 1 \text{ 时}$$

$$T(n) = 2T(n-1) + 1 \quad \text{当 } n > 1 \text{ 时}$$

则：

$$\begin{aligned}
 T(n) &= 2[2T(n-2) + 1] + 1 = 2^2 T(n-2) + 1 + 2^1 \\
 &= 2^3 T(n-3) + 1 + 2^1 + 2^2 \\
 &= \dots \\
 &= 2^{n-1} T(1) + 1 + 2^1 + 2^2 + \dots + 2^{n-2} \\
 &= 2^n - 1 \\
 &= O(2^n)
 \end{aligned}$$

3、采用顺序查找方法，在长度为 n 的一维实型数组 $a[0..n-1]$ 中查找值为 x 的元素，即从数组的第一个元素开始逐个与被查值 x 进行比较，找到后返回 1，否则返回 0。对应的算法如下：

```
int Find(double a[], int n, double x)
{
    int i=0;
    while(i<n)
    {
        if (a[i]==x) break;
        i++;
    }
    if(i<n) return 1;
    else return 0;
}
```

回答以下问题：

(1) 分析该算法在等概率情况下成功查找到值为 x 的元素的最好、最坏和平均时间复杂度。

解：该算法的 while 循环中 if 语句是基本语句。 a 数组中有 n 个元素，当第一个元素 $a[0]$ 等于 x 时基本语句仅执行一次，此时呈现最好的情况，即 $G(n) = O(1)$ 。

当 a 中的最后一个元素 $a[n-1]$ 等于 x 时基本语句执行 n 次，此时呈现最坏的情况，即 $W(n) = O(n)$ 。

对于其他情况，假设查找每个元素的概率相同，则 $P(a[i]) = 1/n (0 \leq i \leq n-1)$ ，而成功找到 $a[i]$ 元素时基本语句正好执行 $i+1$ 次，所以：

$$A[n] = \sum_{i=0}^{n-1} \frac{1}{n} (i+1) = \frac{1}{n} \sum_{i=0}^{n-1} (i+1) = \frac{n+1}{2} = O(n)$$

(2) 假设被查值 x 在数组 a 中的概率是 q ，求算法的平均时间复杂度。

当被查值 x 在数组 a 中的概率为 q 时，算法的执行有 $n+1$ 种情况，即 n 种成功查找和一种不成功查找。

对于成功查找，假设是等概率情况，则元素 $a[i]$ 被查找到的概率 $P(a[i])=q/n$ ，成功找到 $a[i]$ 元素时基本语句正好执行 $i+1$ 次。

对于不成功查找，其概率为 $1-q$ ，不成功查找时基本语句正好执行 n 次。

所以：

$$\begin{aligned} A(n) &= \sum_{I \in D_n} P(I) * T(I) = \sum_{i=0}^n P(I) * T(I) \\ &= \sum_{i=0}^n \frac{q}{n} (i+1) + (1-q)n = \frac{(n+1)q}{2} + (1-q)n \end{aligned}$$

如果已知需要查找的 x 有一半的机会在数组中，此时 $q=1/2$ ，则

$$A(n) = [(n+1)/4] + n/2 \approx 3n/4$$

得分	
----	--

二、简答题（每小题 5 分，共 20 分）

1、算法设计的基本步骤？

答：

(1) 分析求解问题：确定求解问题的目标（功能）、给定的条件（输入）和生成的结果（输出）

(2) 选择数据结构和算法设计策略：设计数据对象的存储结构，因为算法的效率取决于数据对象的存储表示。算法设计有一些通用策略，例如迭代法、分治法、动态规划和回溯法等，需要针对求解问题选择合适的算法设计策略。

(3) 描述算法：在构思和设计好一个算法后必须清楚、准确地将所设计的求解步骤记录下来，即描述算法。

(4) 证明算法的正确性。

(5) 算法分析：同一问题的求解算法可能有多种，可以通过算法分析找到好的算法。一般来说，一个好的算法应该比同类算法的时间和空间效率高。

2、一般来说，能够用递归解决的问题应该满足哪些基本条件？

答：

（1）需要解决的问题可以转化为一个或多个子问题求解，而这些子问题的求解方法与原问题完全相同，只是在数量规模上不同。

（2）递归调用的次数必须是有限的

（3）必须有结束递归的条件来终止递归

3、简述动态规划法与分治法的异同？

答：

动态规划的基本思想与分治法类似，也是将待求解的问题分解为若干个子问题（阶段），按顺序求解子问题，前一子问题的解为后一子问题的求解提供了有用的信息。但分治法中各个子问题是独立的（不重叠），动态规划适用于子问题重叠的情况，也就是各个子问题包含公共的子子问题。

4、简述用贪心法求解的问题应具有的性质？

（1）贪心选择性

所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择达到。

（2）最优子结构性

当一个问题最优解包含其子问题的最优解时，称此问题具有最优子结构性。

得	
分	

三、算法设计题（每小题 15 分，共 30 分）

1、给定 k 个排好序的序列 $s_1, s_2, \dots, s_k$ ，用 2 路合并算法将这 k 个序列合并成一个序列。假设采用的 2 路合并算法合并 2 个长度分别为 m 和 n 的序列需要 $m+n-1$ 次比较。试设计一个算法确定合并这个序列的最优合并顺序，使所需的总比较次数最少。为了进行比较，还需要确定合并这个序列的最差合并顺序，使所需的总比较次数最多。请给出算法设计的思路，并写出相应的伪码。

贪心设计策略如下：

- (1) 最多比较次数的合并方案，每一次合并的时候都是该序列中最长的那两个。
- (2) 最少比较次数的合并方案，每一次合并的时候都是该序列中最短的那两个。

```
int a[1000];    //升序数组
int b[1000];    //降序数组
bool cmp(int a, int b) { return a > b; }

int MaxMerge(int a[], int k) {
    int max = 0;
    for (int i = 0; i < k - 1; i++) {
        sort(a + i, a + k, cmp); //降序排列 只对后面还没有操作的元素进行重新排序
        a[i + 1] = a[i] + a[i + 1];
        max = max + a[i + 1] - 1;
    }
    return max;
}

int MinMerge(int b[], int k) {
    int min = 0;
    for (int i = 0; i < k - 1; i++) {
        sort(b + i, b + k); //升序排列
        b[i + 1] = b[i] + b[i + 1];
        min = min + b[i + 1] - 1;
    }
    return min;
}

int main() {
    int k;
    cin >> k;
    int x;
    for (int i = 0; i < k; i++) {
        cin >> x;
        a[i] = x;
    }
}
```

```

        b[i] = x;
    }
    cout << MaxMerge(a, k) << " " << MinMerge(b, k) << endl;
    return 0;
}

```

2、设计一个算法，采用分治法求一个整数序列中的最大和最小元素。请给出算法设计思路，并写出相应的伪码。

解：

情况 1，如果数组 $a[\text{low} \cdots \text{high}]$ 只有一个数据，那么最大的数 maxe ，最小的数 mine 都为 $a[\text{low}]$ 。

情况 2，如果数组 $a[\text{low} \cdots \text{high}]$ 有两个数据，那么比较两个数的大小。

情况 3，如果数组 $a[\text{low} \cdots \text{high}]$ 有 3 个及以上数据，那么采用自顶向下的二路归并思路，设置一个 $\text{mid} = (\text{low} + \text{high}) / 2$ 将数组 $a[\text{low} \cdots \text{high}]$ 一分为二，递归地对两个子序列 $a[\text{low} \cdots \text{mid}]$ 和 $a[\text{mid} + 1 \cdots \text{high}]$ 继续分解，结束条件为子序列长度为 1。

求出左区间最大元素和最小元素 $\text{lmaxe}, \text{lmime}$ 和右区间最大元素和最小元素 $\text{rmaxe}, \text{rmime}$ 。

然后，合并子问题的解。如果左区间最大元素大于右区间最大元素，那么 $\text{maxe} = \text{lmaxe}$ ，然后比较左区间最小元素和右区间最小元素的大小。

```
#include <stdio.h>
```

```
#define max(x,y) ((x)>(y)?(x):(y))
```

```
#define min(x,y) ((x)<(y)?(x):(y))
```

```
void MaxMin(int a[],int low, int high, int& maxe, int &mine)
```

```

{
    if(low == high)
    {
        maxe = a[low];
        mine=a[low];
    }
    else if(low == high -1 )
    {
        maxe = max(a[low],a[high]);
        mine=min(a[low],a[high]);
    }
    else
    {
        int mid=(low+high)/2;

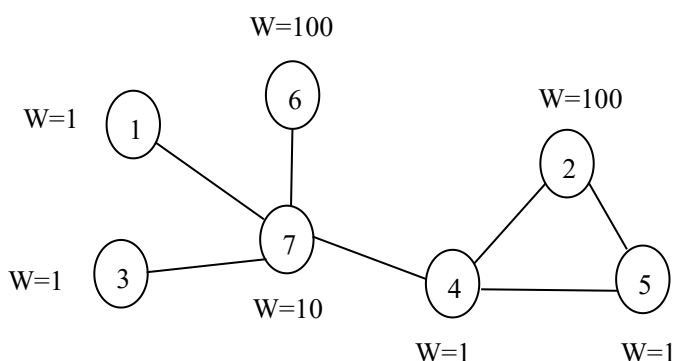
```

```
int lmaxe,lmine;  
MaxMin(a,low,mid,lmaxe,lmine);  
int rmaxe,rmine;  
MaxMin(a,mid+1,high,rmaxe,rmine);  
maxe = max(lmaxe,rmaxe);  
mine = min(lmine,rmine);  
}  
}
```


得	
分	

四、算法分析题（每小题 10 分，共 20 分）

1、给定一个赋权无向图 $G=(V,E)$ ，每个顶点 $v \in V$ 都有权值 $w(v)$ 。如果 $U \subseteq V$ ，且对任意 $(u,v) \in E$ 有 $u \in U$ 或 $v \in U$ ，就称 U 为图 G 的一个顶点覆盖。 G 的最小权顶点覆盖是指 G 中所含顶点权之和最小的顶点覆盖。对于给定的无向图 G 如下图所示，计算 G 的最小权顶点覆盖，给出具体算法设计思路。



解：

设计一个优先队列式分支限界法，计算 G 的最小权顶点覆盖。算法设计思路如下：

(1) 优先级：顶点权重小的顶点（由于所有顶点都是带权的，我们的目的也是找到最小权覆盖，所以我们可以直接用权重作为优先级建立一个最小堆，从而实现优先队列）。

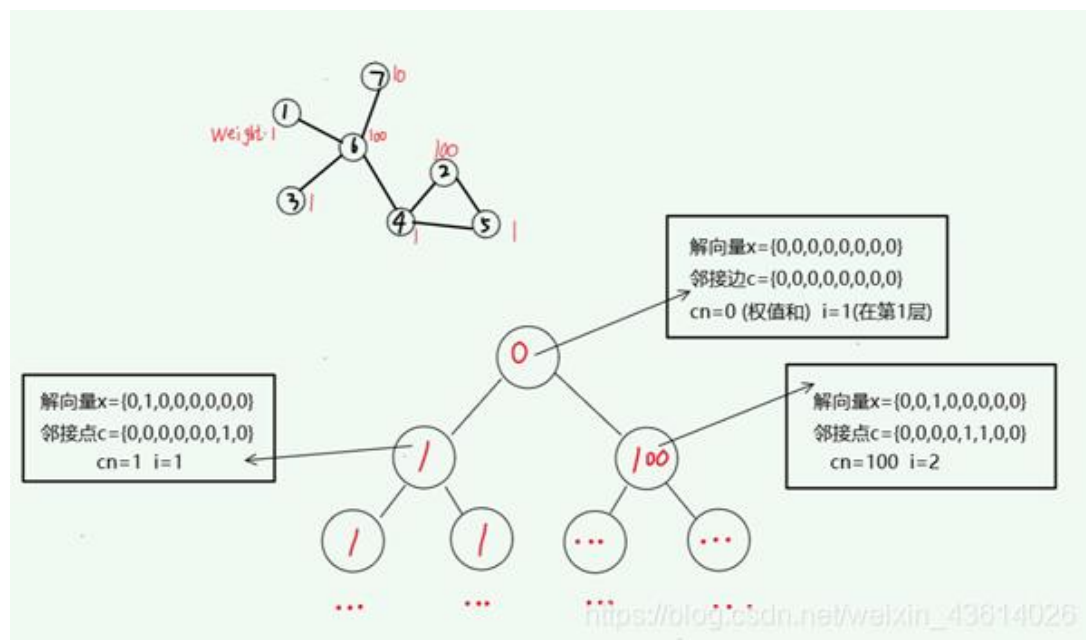
(2) 限界函数：我们找的是最小点权，无法使用限界函数来对右孩子进行约束，因为如果当前结点不加入 U 集合中(即走右孩子路径)，一定顶点权值之和更小，但是不一定满足覆盖条件，所以不经过判断，我们也要将右孩子加入到队列中。

(4) 解向量： $x=\{y_1,y_2,y_3,y_4,y_5,y_6,y_7\}$ ， $y_i=0$ 或 1 ， $y_i=0$ 表示顶点 i 不在集合 U 中， $y_i=1$ 表示顶点 i 加入到了集合 U 中。

(5)剪枝策略：要找到最小权覆盖，要先判定该点能否加入集合，也就是判断集合 V 中的点是否能与 U 集合中的任意一点直接相连，如果当前方案不是一种顶点覆盖，直接进行剪枝。

算法思路：每个顶点都有两种选择，加入 U 集合或者不加入 U 集合，显然进入右子树的选择一定是比进入左子树的选择所得的权值和小，但是进入右子树不一定满足顶点覆盖，所有当试探完左子树的结果后，还需进入右子树进行测试。将每个试探的解向量

加入到顶点权值小优先的优先队列，每次取出堆顶的解向量判断能否满足完全覆盖，如果能则得出最小权覆盖的解，因为堆顶解向量满足顶点权值和最小的原则，如果不能则继续取堆顶解向量判断。



2、给定一个无序的整数序列 $a[0..n-1]$ ，要求使用动态规划算法设计策略求其中最长递增子序列的长度。例如， $a[] = \{2, 1, 5, 3, 6, 4, 8, 9, 7\}$ ， $n = 9$ ，其最长递增子序列为 $\{1, 3, 4, 8, 9\}$ ，结果为 5。

解：

设计动态规划数组为一维数组 dp ， $dp[i]$ 表示 $a[0..i]$ 中以 $a[i]$ 结尾的最长递增子序列的长度，对应的状态转移方程为：

$$dp[i] = 1 \quad 0 \leq i \leq n-1$$

$$dp[i] = \max(dp[i], dp[j] + 1) \quad \text{若 } a[i] > a[j], 0 \leq i \leq n-1, 0 \leq j \leq i-1$$

求出 dp 后，其中的最大元素即为所求。

```
1  int a[] = { 2,1,5,3,6,4,8,9,7 };
2  int n = sizeof(a) / sizeof(a[0]);
3  int ans = 0;
4  int dp[MAXN];
5
6  void solve(int a[], int n)
7  {
8      int i, j;
9      for (i = 0; i < n; i++)
10     {
11         dp[i] = 1;
12         for (j = 0; j < i; j++)
13         {
14             if (a[i] > a[j])
15                 dp[i] = max(dp[i], dp[j] + 1);
16         }
17     }
18     ans = dp[0];
19     for (i = 1; i < n; i++)
20         ans = max(ans, dp[i]);
21 }
```

